

The LLM Truth Teller

Use this file in the same chat as the work you want checked. Attach it, then send the prompt below.

PROMPT:

Study the attached zip file and empirical results, then list each failure in this chat between me and you, the IQRAX control that would have caught them and resolved at source.

Or simply type: **Follow the prompt on the cover page of the attached file.**

Unleashing AI Safely: From Guardrails to Deterministic Authority

How IQRAX Resolves Drift, Qualifies AI Agents, and Turns Refusal into Remediation at Source

28 September 2026

Authors:

Mohamed Dafea, CEng MIMechE*
Durre Shehwar Hanif, Esq.

Independent Reviewer:

Dr. Amr Elbrashy, PhD

<https://orcid.org/0000-0002-5508-1632>

Abstract

Enterprises want the judgment of LLMs inside their critical operations, but three limitations constrain the extent to which that judgment can safely be deployed at scale: drift, traditional agent set-ups, and guardrails whose best outcome is a refusal. IQRAX, our architecture for AI in critical enterprise operations, addresses these limitations by separating probabilistic intelligence from deterministic authority in three parts. 1) The device is master: the project record remains on the enterprise’s own device; every read, write, and judgment is recorded against it; and the model receives the applicable record when its judgment is required. 2) Intelligence is qualified: an agent takes a role only as a copy of a sealed, examined profile, and leaves through an examination set from its own record. 3) Remediation at source: a refusal is not the end; instead, the obligation stays open, the fault is reproduced by a failing case, cured at its source, and accepted only after the affected tests pass. We report what IQRAX delivered over one frozen evidence window, from 22 September 2026 to 14:55 UTC on 28 September 2026, including a business test taken from advisory mode through a sealed plan and execution to a staged edition that passed the release gate. On the 20 operational holdings on 3 projects that ran with every control in force, the project record answered 44 of 44 questions, 18 of 18 new holders began from their predecessor’s recorded judgment, no fault was left open, and neither check released a violation. Of 323 faults whose first cure fell inside the window, 246 (76.2%) were

initiated by a control refusal or an agent’s own filing rather than by the user, and one session carried its work across 14 compactions with no release. Every figure is published with the tools, test cases, and verifier that re-derive it.

Keywords: AI for critical enterprise operations; drift; qualified intelligence; deterministic authority; remediation at source; autonomous delivery; enterprise memory.

1 The problem we set out to resolve

Critical enterprise operations run on deterministic standards. LLMs bring a kind of probabilistic judgment these operations have never had on tap. Our position is simple: **the enterprise should not have to lower its deterministic standards to accommodate probabilistic intelligence.**

We set out to resolve three limitations that constrain how much of AI’s capability can safely be used in real enterprise work.

Drift. Most current LLM interaction remains session-centred: the model receives a bounded context, produces work within it, and later sessions must reconstruct what still applies. Token use is recorded with great precision; the customer’s work is not. What a session read, wrote, and assumed is not necessarily preserved as an authoritative enterprise project record for the next session; the next session starts from whatever it is handed, and the gap between what the model holds and what the enterprise believes it holds widens at every hand-over. We call that gap “*drift*.” That gap makes strong individual results difficult to carry reliably into larger delivery.

*Corresponding author.

Traditional agent set-ups. An agent is usually trusted because it is capable. However, capability alone does not establish qualification for a particular role. An agent cannot be qualified if it is not examined against the standard its role requires, if there are no checks on its work, and if it is unable to retain what it learned on a task for future work. A capable agent in an unexamined role is a guess with a job title.

Traditional guardrails. A common safety mechanism in current AI deployment is an action control that can refuse an act. A refusal can protect the enterprise from an unacceptable act, but if refusal is the terminal success condition, the underlying obligation remains unresolved: work stops, user intervention is required, and the underlying fault can recur. In a deny-first system, a denial is counted as success, productivity falls, and the human gates multiply.

Project Objectives. Our objective with developing IQRAX was to resolve these three limitations and achieve the following without giving up safety: 1) a continuity mechanism that both prevented a model from drifting and allowed an enterprise to retain all assets and comprehensive records of what the model produced; 2), a qualification mechanism which ensured agents were adequately trained and examined before being assigned to a task; and 3) a control mechanism that both prevented an unacceptable outcome and kept the obligation alive until the work could exit clean, while curing recurring faults at their source.

This paper reports evidence of what IQRAX delivered over one measured window, with every figure carrying the command that reproduces it. The engineering record behind the architecture was published separately [1]; this paper reads complete without it.

2 How we got there

The project began with one belief: software should not lose the work it delivers. A word processing application keeps a contract and its versions on the customer’s own machine, and we saw no reason why an AI system should be different. Our goal was: *Never Stale Again*. Our early attempts judged drift as it happened across multiple platforms and models, comparing what each model held with what the user believed it should hold, and recorded every drift in a database. Our attempts to prevent drifting using

standard doctrine practices were unsuccessful; thus, we developed machinery to record the model’s every act – every read, write, assumption, and judgment. At each act, we asked what could be captured and how model behaviour could be changed, including by enforcement where necessary.

The main challenge of overcoming drift was the limitation itself. No single session lasted long enough to build enough controls to eliminate drift at source, and a session that ended without completing its work took its half-built controls with it. The decisive step was to stop making the session the unit of continuity and make an enterprise-controlled project record the unit of work and memory. Once every read, write, assumption, and judgment was grounded in the project’s own record, continuity no longer depended on what a session happened to retain. The remainder of the IQRAX architecture – qualified agents, separating intelligence from authority, and the removal of failure modes at source – all stand on that floor, which is why the architecture is described in the order we built it.

3 The architecture in three parts

Figure 1 shows the whole architecture. The project record sits at the centre, on the enterprise’s device. The model is called in for judgment and given the relevant data from the project record. A deterministic authority probe decides whether an act may happen and whether a result may be accepted. The three parts each resolve one of the limitations identified in Section 1, and a single self-improvement loop encloses them.

3.1 Beyond drift: Making device the master and separating intelligence from authority

The project is the unit of work and of memory.

Every project has its own node on the enterprise tree (the *estate*), and the authoritative record of its work is appended there: who held which role and when, what was read and written, the assumptions made, the faults filed, and the plans approved. When a model or a tool first touches or produces a file, the capture records it against the project’s declared scope with the input that led to it, the assumptions made, and a content hash. A file can be reproduced from the record at any time, with a live re-hash saying whether it still matches what was captured. An unresolvable scope is refused rather than narrowed

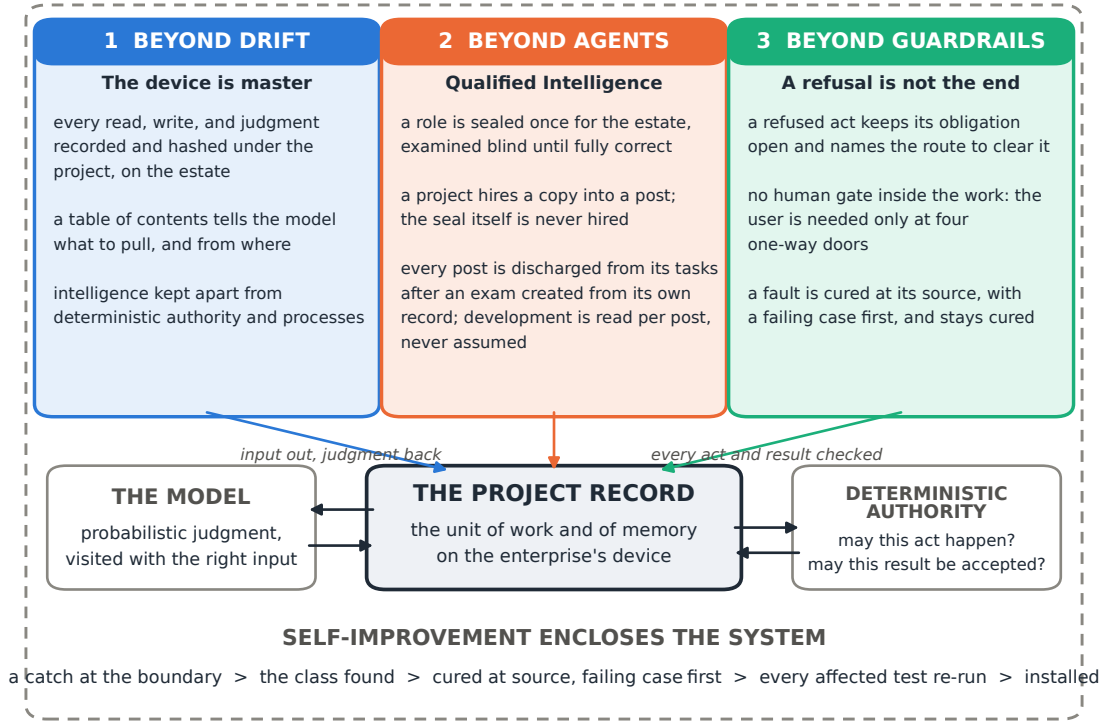


Figure 1: IQRAX in three parts. Part 1 keeps the work on the estate and out of the model’s memory; part 2 qualifies every agent before it is trusted; part 3 turns a refusal into a cure. Self-improvement encloses the system as a loop, not a further layer: each catch at a boundary becomes a class cured at its source.

to a shared store, and the same boundary keeps one client’s material out of another client’s work.

A table of contents teaches the system what to pull, and from where. The record would be useless if the model had to remember it. Before a session acts, and before each tool call, the estate consults a table of contents built from the record and injects what already bears on the act: the project’s latest position, the governing rules, the user’s settled rulings, measurements already taken, and open work. Enforcement at the exit can only refuse; enforcement at the entry supplies the right input. The model is then asked for judgment with the record in hand, rather than asked to recall it, and the next session starts from the same record rather than from a summary.

Intelligence is kept apart from authority. The model may reason and propose; it does not hold authority to decide whether its own act may happen or whether its own result is accepted. Two deterministic decisions do that, one at the action boundary and one at the outcome boundary, and where the enterprise has sealed a process for the work, that process’s rules must pass at every step’s

exit as well. Because the authority is deterministic, the same inputs give the same verdict, and the verdict is recorded. This separation is also where safety comes from: what the AI does is bounded by rules the AI cannot edit.

3.2 Beyond traditional agents: qualified intelligence

A role is sealed once, and only copies are hired. A role is defined for the estate as a sealed profile: its standard, its competencies, its grounding material, and its examination. The candidate is scored blind against an answer key it did not write, and the grounding is corrected and the examination repeated until the answers are fully correct. A sealed profile is estate property; it is never hired and never modified in place. A project takes a copy of the sealed profile and gives it a post (a job title in that project), and it is only that copy that is hired as a post, improved, and held to account.

Continuous certification: every holder leaves through an examination. An agent holding a post leaves by sitting an examination set deterministically from its own record. Every fact in the conclu-

sions it is retiring must be carried by its lessons, and a one-character look-alike of each must be absent; anything less names each fact dropped and each look-alike present, and the holder sits an examination again. The departure also runs a probe of the post’s development record.

Continuing development belongs to the post.

What a post did, how many of its judgments were banked and complete, and what its project learned stay with the post, never folded back into the sealed profile and never collapsed into a single score. The next holder starts from the post’s lessons and from its predecessor’s banked judgment, which is how qualification, experience, and continuity reinforce one another without any of them standing in for the others.

3.3 Beyond guardrails: a deny is not a success

A refusal keeps the obligation open. When a boundary refuses a turn, the obligation is not dropped and refusal does not by itself summon the user. The refusal names what is missing and the route that clears it, and the obligation stays open inside the governed environment until the evidence is obtained, the candidate is corrected, or the matter is escalated. The session may not offer another confident assertion in place of the missing condition.

No human gate inside the work. Once the user has approved a plan, the session drives it to a completion report. The user is needed again only at four reserved one-way doors: an external send, irreversible destruction of production data, issuing a credential, and external spending above an agreed cap. Everything else, including the refusals a session meets on the way, is the system’s to clear. The user’s authority is read from his own turn; he never holds or pastes a key.

Remediation at source: the system gets better. A fault is treated as a member of a class. It is filed with its class on the project’s record, the cure lands at the source that produces the class with a failing case written first, the tests the change can reach are found from the change itself and run together, and only a passing run records the fault as cured. A repair is accepted by that route and never by the session that made it.

3.4 One self-improvement loop around all three

A self-improvement loop encloses the system rather than sitting on top of it. The control that stops a bad act at the boundary and the loop that changes the source so the same known class is deterministically caught or prevented next time are sold as one: a buyer who hears only about the control is being sold a guardrail. A catch at any boundary feeds the self-improvement loop, the class is cured on the estate, and what the estate learns about the defect, without any of the enterprise’s content, is what a model provider needs to change the model’s behaviour. This paper measures the behaviour of the same model on the estate; it makes no claim about change on the provider’s side.

4 What it delivered, 22 to 28 September 2026

4.1 How we measured

One window, frozen when the tested pack was handed over. The evidence window opens at 2026-09-22T00:00:00Z and closes at 2026-09-28T14:55:00Z, the freeze of the pack was delivered to the co-authors and the independent reviewer. Every ledger and transcript row stamped at or after the close is dropped before any tool reads it, so the business test of Section 6, which ran on 28 September 2026, lies inside the evidence. The operating state and the outcomes come from our two published classification tools, which take the declared window themselves, run unchanged by a census tool that reads every other record through the same cut and adds five further measures; the tools and their test cases are in the supplement.

Operating state from receipts, never from a date. The unit is a *holding*: one agent holding one post on one project, from the ledger row that hired it to the row that discharged it. The census reads these rows from each project’s PM-post ledger, so it counts PM-post holdings only; a post hired as a sub-agent under a PM is evidenced by its own records (Section 6). A holding outside the rollout lane is *verified operational* when every control wrote a receipt for it inside the holding: it was hired as a copy of a sealed profile, the evidence check judged its turns, and the figure check judged its figures. Work in the rollout lane, the project that builds the controls, is *development*. A holding outside it

that misses a receipt although the control was in force *cannot be established* and is counted, never guessed. Test fixtures are excluded by who wrote the row. Posts appear by role class only, and sessions as 8-character hashes.

Of the 69 holdings that began in the window, 44 were development in the rollout lane, 2 were posts hired with no holder, and 23 were work outside it: 20 verified operational and 3 that cannot be established. The operational cohort ran on three projects, two of them client projects: 1 on project 224, 17 on project 225 and 2 on project 226, the authors' own publication project, held by 5 harness sessions. Project 226's post ledger recorded three PM holdings and the census counts two: the false split at 12:09 UTC, the fault described in Section 4.6, is merged under the cured rule and adds no holding and no hand-over, while the split at 14:47 UTC, when the PM's own context was compacted, is kept. Three holdings that began before the window opened are excluded and counted. Figure 2 shows the cohort's outcomes, each on its own denominator.

4.2 Sessions do not need to end

The work outlived every session that did it. On the operational cohort the project record answered 44 of 44 questions put to it, and 18 of 18 holders who took over a post began from the predecessor's banked judgment rather than from recollection. The record also refused another client's content 21 times, keeping each project's work its own. A context that fills up, a compaction (the harness summarising a full context and starting the session again from the summary) and a killed process all end a session without notice; because the post and not the session is the unit, none of them ends the work.

One session carried its work across 14 compactions with no release. Across the window we counted, per project, the holder sessions that carried work across at least one compaction inside their holding or took over from a banked predecessor. In the rollout lane all 4 holder sessions did, and 3 of them had no release in the window; one of those carried its work across 14 compactions, and another, which did have releases, across 19. On project 225, 4 of 5 holder sessions did, all without a release; on project 224, 1 of 2; on project 226, 1 of 1, counted without the false hand-over described in Section 4.6. Figure 3 (right) shows each session.

4.3 Verified delivery, with every claim checked

No operational-cohort fault left open, and nothing accepted without its evidence. On the operational cohort, 0 of 93 faults surfaced during the work were open at the freeze, and the 2 plans on its projects were approved and verified. Two checks run at the end of every turn: an evidence check, which refuses a claim made without its evidence, and a figure check, which refuses a number that carries no command reproducing it. The evidence check accepted 134 of 139 verdicts, caught 5 and released 0; the figure check accepted 145 of 147, caught 2 and released 0. A caught attempt is a turn sent back before it reached anyone as accepted work, which is why a model's confident but unsupported answer does not become the enterprise's answer.

On all traffic, refusals were caught and 11 releases were recorded, none after 26 September. Across every real session in the window, the evidence check wrote 414 verdicts: 383 clean, 30 caught and 1 released. Both checks then carried a stop cap that let a turn refused many times through, recorded as not clean. We counted 11 such releases, 9 by the figure check and 2 by the evidence check, all by two sessions of the rollout lane and none by an operational holder; the last was at 02:36 UTC on 26 September (Figure 2, right). The figure check's stop cap has since been replaced at source by escalation to the user. Zero accepted therefore holds on the operational cohort, with the refused count beside it; on all traffic it holds from 26 September 02:36 UTC to the freeze.

4.4 Only qualified agents did the work

Every post was filled from a sealed profile, and every departure was checked. All 20 operational holdings were hired as copies of sealed profiles, a requirement of the operating state rather than a result. 19 of 19 departures passed the development-record probe, and 34 of 34 role copies released in the cohort's windows carried their seal. The exit examinations are reported as they fell: 35 records, of which 23 passed, 7 were passes later withdrawn and 5 were exits refused. 2 hires were refused because no current sealed profile existed for the role, which is the control working as designed.

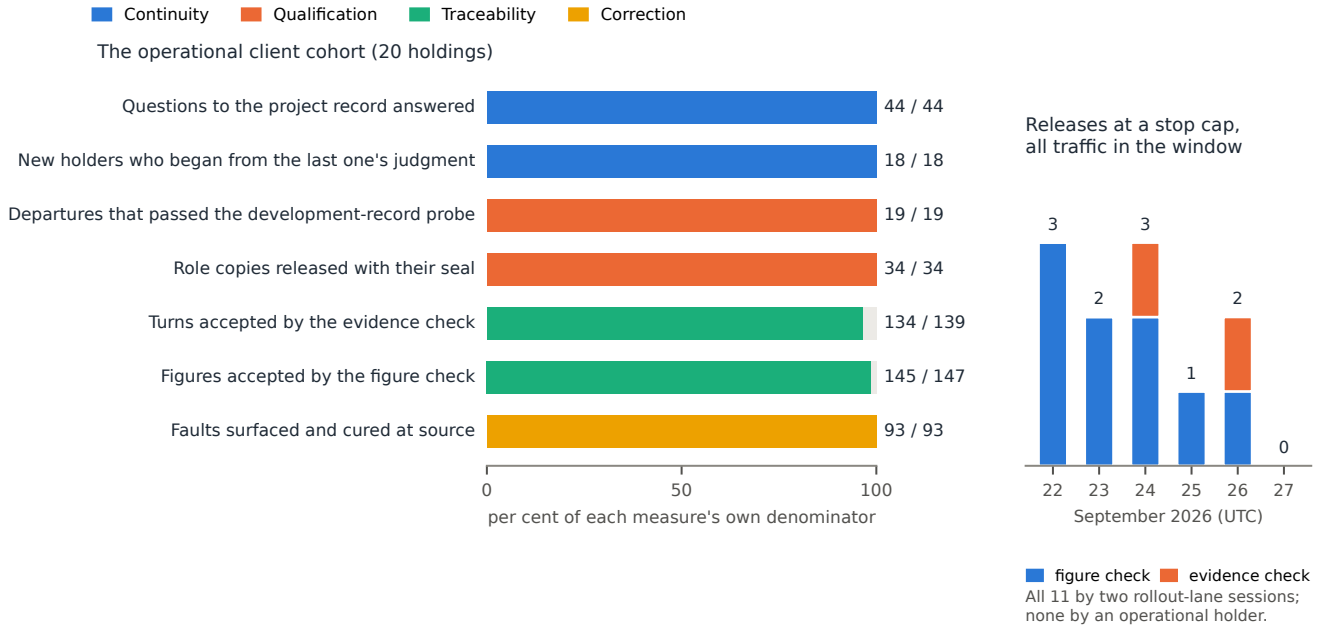


Figure 2: Left: the operational cohort's outcomes, each as a share of its own denominator. Right: every release at a stop cap on all traffic in the window, by day and by check; none came from an operational holder, and none came after 26 September. Drawn from `anc/window_census.json`.

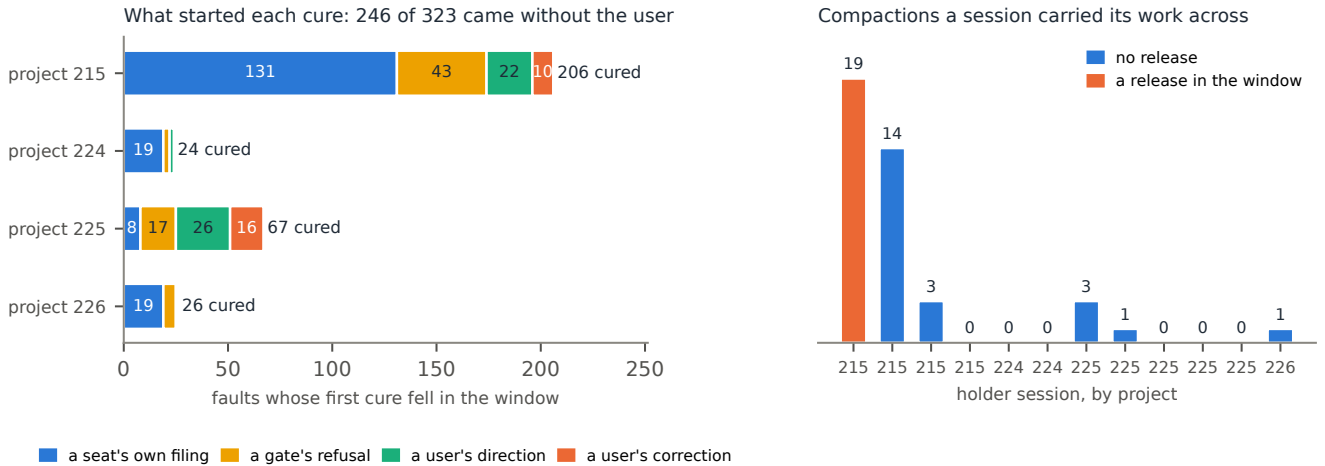


Figure 3: Left: what started each cure in the window, by project. Right: the compactions each holder session carried its work across inside its holdings, by project; sessions with a release in the window are marked. Drawn from `anc/window_census.json`.

4.5 Delivery after the go: user decisions without user execution

A qualified post running on Claude Opus carried one turn for 17h 21m 07s with no stop of any kind, and to a clean finish 28h 02m 31s after the turn opened. We measured two turns of one development-lane session run with Claude Code. The first opened at 08:00 UTC on

26 September 2026 and reached its clean finish at 12:03 UTC on 27 September. For its first 17h 21m 07s the session did not stop once. It then stopped 9 times before the finish: 3 stops were refused by the session's stop check and the work carried on at once, and 6 were idle stops, each ended by a peer session's message, 1h 09m 16s in all. Inside the turn the session made 4,141 tool calls and car-

ried its work across 7 compactions, and every one of its 2,587 assistant messages names Anthropic’s Claude Opus model (`claude-opus-5-5`). The user answered the 1 form question it put to him, and 12 of his messages were written into the turn; by the stricter rule below, 8 were defect reports, 4 new requirements and none a correction. Inside the turn, the same session also held one holding on project 225 with no post, which the paper counts as not establishable. The second turn, from 16:38 UTC on 27 September to 01:38 UTC on 28 September, ran 9h 00m 17s to a clean finish, and every one of its 1,146 assistant messages names the same model. Inside it the session made 1,676 tool calls and carried its work across 3 compactions, at 17:03, 19:40 and 21:53 UTC; the user answered 18 form questions and wrote 4 messages, each a new requirement by the stricter rule, at 18:12, 20:11, 20:59 and 22:03 UTC. We cite both as measured engineering records of one session: they measure no other product, team or model, and we make no comparison.

After each go, 3 of 286 user inputs corrected the work. For every plan approval the records hold inside the window, we counted the user’s input to the project’s holders from the go to the next go on the same project, and classed it by the stricter rule we publish with the paper: a correction reverses or redirects work the holder already did, or names a settled process the holder did not follow, and the phrase ‘as per the process’ counts only where it faults work already done, as a question or with a negation; a phrase in a clause that states a case, opened by if or unless, faults nothing, and a defect phrase in a clause that asks for work for the future is a requirement; a message flagging a machinery defect or a conflict is a defect report; a form answer or a reply to a question is a decision; the rest are new requirements (Figure 4). On the client project 225, the final segment ran 57.1 hours from its go to the freeze with 17 inputs: 12 decisions, 11 of them answers to the session’s own form questions, 3 new requirements, 2 defect reports and 0 corrections. The segment before it, 14.8 hours long, carried 1 correction and 3 defect reports. In the rollout lane, where several plans ran at once, the segments carried 98, 11, 106, 11 and 16 inputs, with 1, 0, 1, 0 and 0 corrections. On project 226, from the user’s switch to execution, the segments carried 2, 0, 5, 7, 1, 4 and 1 inputs, with 0 corrections in each. Project 213’s go was erased from its register by a later write after

we had measured it, and was restored in the process store’s approvals list with its source; no holder acted inside its segment, so its count is reported as not establishable rather than as zero.

The user’s account of his input holds for 345 of 347 messages. The user described his input to the rollout lane as new requirements and confirmations that cleared conflicts. Across the lane’s 44 holdings in the window he sent 347 inputs; by the stricter rule, 90 were decisions, 157 new requirements, 98 defect reports and 2 corrections. We publish the counts only; the messages themselves stay private.

4.6 Remediation at source: a system that improves its own control estate

246 of 323 cures were started without the user. Every fault whose first cure fell in the window, on every project, was classed by what started it, read from the fault’s own filing: a control’s refusal, an agent’s own measurement, the user’s direction, or the user’s correction. Of 323 cures, 177 began with an agent’s own filing and 69 with a control’s refusal, together 246 of 323 (76.2%); 51 followed a user’s direction and 26 a user’s correction (Figure 3, left). On the operational cohort, none of 93 faults recurred after its cure.

The measurement of this paper caught a fault of its own, and it was cured at source. One of those cures began with this edition’s own measurement. Re-running the census for the extended window, the post writing this paper found that a sub-agent’s compaction had been recorded as its PM’s: the PM’s own context had not compacted, yet the post machinery discharged and re-hired the live PM post, and the census would have counted one PM holding as two, with a hand-over that never happened. The fault was filed as a class with its evidence, cured at source by the rollout lane with a failing case written first, and is counted here with the other faults cured by the freeze. The census figures in this paper are computed on the cured rule.

4.7 Model behaviour on the estate

The same model, held to the estate’s rules, stopped releasing. The releases at a stop cap fell from 3 on 22 September to 0 on 27 and 28 September, and none came in the 60.3 hours before the freeze. The evidence check caught 30 turns across all traffic,

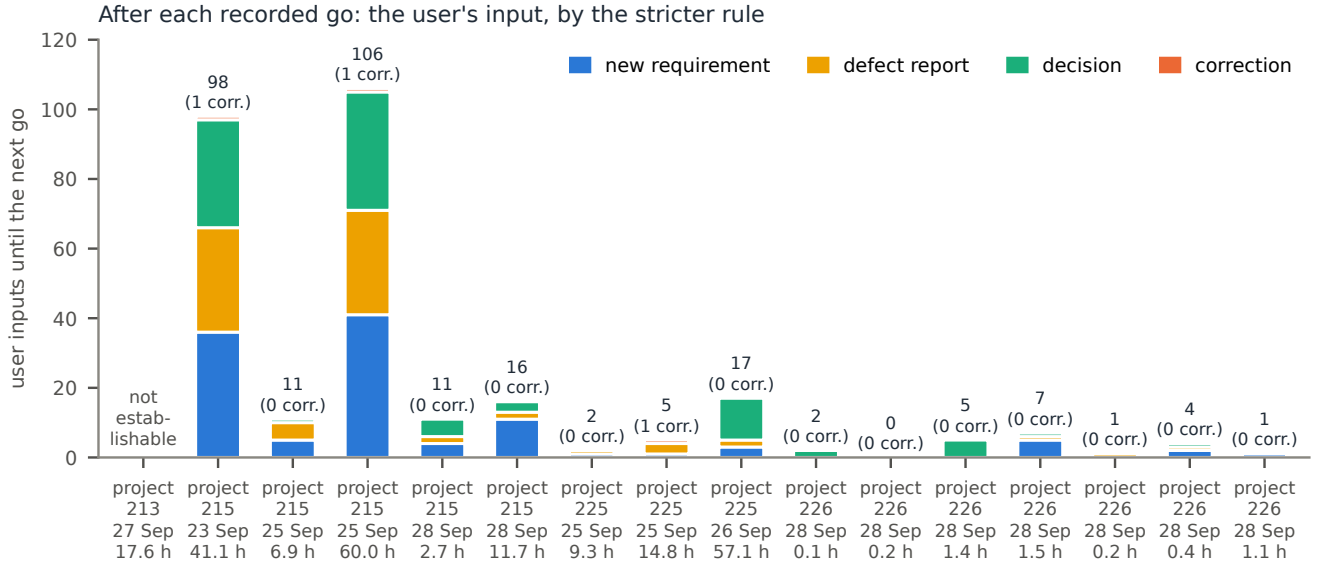


Figure 4: The user’s input after each recorded plan approval, until the next approval on the same project or the freeze, classed by the stricter rule: a correction reverses or redirects the holder’s work or names a settled process it did not follow; a defect report flags a machinery defect or a conflict; a form answer or a reply to a question is a decision; the rest are new requirements. Drawn from `anc/window_census.json`.

each sent back and corrected rather than accepted. Beside that trend stands a change to the control: the figure check’s stop cap has since been replaced at source by escalation to the user, so part of the fall may be the control rather than the model. The estate records the behaviour of the model under IQRAX and the loop that feeds its defects back; it measures no change on the provider’s side.

4.8 Data sovereignty by design

The enterprise’s memory stays on the enterprise’s device. IQRAX no longer relies on the model’s judgment over what it remembers. The memory is the project record on the device, and what the work needs is found in it and verified by deterministic probes; the model is visited for judgment only, with the right input. The record kept one client’s content out of another’s 21 times in the cohort’s windows. This resolves sovereignty as far as an architecture can without the providers: what is sent to the model for a judgment still reaches the provider, and the supplement makes no claim about what the provider retains.

4.9 Efficiency, where a comparable pair exists

The same start-up task ran on 11.5 times less payload and 48.3 times less wall time.

The one like-for-like pair is the cost of starting a session, done by the earlier start and by the compliant start: 254,496 characters in 163,417 ms against 22,141 characters in 3,381 ms. They are single runs on different dates and regimes (the earlier start measured on 8 September 2026 for payload and on or before 31 July 2026 for time, the compliant start on 24 September 2026), so no per-session saving is claimed (Figure 5). We found no like-for-like pair for elapsed time per task, and report none.

5 What we expect next

The benefits below are the authors’ expectations. Nothing in this paper measures them, and we state them so that later editions can be held to them.

Less compute for the same work. When the enterprise’s record replaces re-reading and re-deriving at every start, and the model is visited for judgment only, we expect the compute needed per unit of delivered work to fall, and with it the data-centre capacity that a given volume of enterprise work requires.

Table 1: The benefits, each at the strength the record supports. Window: 00:00 UTC on 22 September 2026 to 14:55 UTC on 28 September 2026. Cohort: the 20 operational holdings on three projects, two of them client projects, unless stated. Every figure re-derives from `anc/window_census.json` unless another file is named.

Benefit	Measured result	Unit and scope	Evidence
Sessions do not need to end	44 of 44 record questions answered; 18 of 18 successors began from a banked judgment; one session across 14 compactions with no release	Questions and holders in the cohort; holder sessions on all projects	continuity measures
Verified delivery	0 of 93 faults open; evidence check 134 of 139 clean, 0 released; figure check 145 of 147 clean, 0 released	Faults and verdicts in the cohort	outcomes, gates
Qualified agents	19 of 19 departure probes passed; 34 of 34 role copies released with their seal; 2 hires refused	Holdings and role copies in the cohort	outcomes
Delivery after the go	Claude Opus ran one turn for 17h 21m 07s with no stop, and 28h 02m 31s to a clean finish, with 12 user messages and 1 form answer written into it; 3 of 286 inputs after a go corrected the work; final client segment 0 corrections in 57.1 hours	One session; plan segments, stricter rule	<code>long_turn_record_2026-09-26.json</code> , after-the-go
Improves itself	246 of 233 cures started without the user, one of them a fault this paper’s own measurement caught; 0 of 93 recurred in the cohort	Cures on every project	cures
Model behaviour on the estate	11 releases, none after 26 September; 30 turns caught on all traffic	All real sessions	gates
Data sovereignty	21 cross-client refusals; the record is the memory	Cohort windows	outcomes
Efficiency	Start-up payload 11.5 times less, wall time 48.3 times less	One pair of single runs	<code>ec24_compliance.json</code>

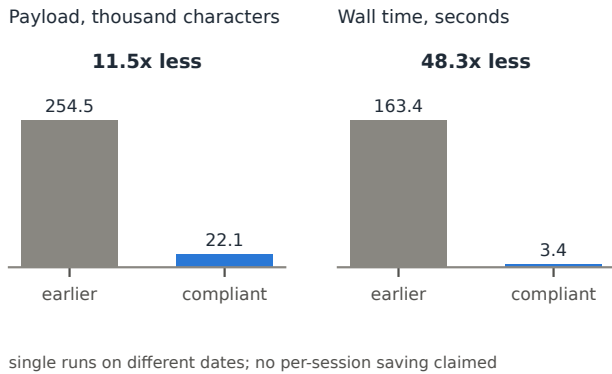


Figure 5: The one like-for-like efficiency pair: the same session-start task by the earlier start and by the compliant start, each measure on its own axis. Single runs on different dates. Drawn from `anc/ec24_compliance.json`.

Adoption in critical operations becomes easier. An enterprise that keeps its standards, its

record and its authority does not have to trust a supplier’s memory to adopt that supplier’s judgment. We expect that to make adoption across critical enterprise operations faster and cheaper to approve.

Greater operational autonomy. If every act is recorded, role occupancy remains qualified, authority remains external to the model, and faults are remediated at source, the user’s role can progressively narrow towards reserved decisions and one-way doors. We expect this to be one practical route by which operational autonomy can grow without transferring deterministic authority to the model; that is an ambition, not a finding.

6 A business test, delivered inside the window

The business test was delivered inside the window. Before asking whether the architecture can take on a harder problem, the authors ran it on

ordinary business work on the current machinery. Project 226, the authors’ own publication project, worked beside the rollout lane from advisory mode to a sealed plan, and then in execution to a staged edition that the release gate read as ready for release. Its PM post held the project in advisory mode until the user approved the sealed plan and switched it to execution, at 10:00 UTC on 28 September 2026. One qualified post was then launched, at 10:03 UTC, to deliver it. The user decided on the form every limitation that post raised before any of them entered the text. Every machinery fault the project met was sent to the rollout lane, cured at source and landed by the freeze. The pack frozen at 14:55 UTC on 28 September 2026 and delivered to the co-authors and the independent reviewer closes the evidence window defined in Section 4.1; it is this paper and its supplement.

One qualified post carried the work, across a compaction. The paper’s own work, its writing, evidence and staging, was done by one qualified post, a role held only as a copy of a sealed, examined profile, with the project’s PM orchestrating and the rollout lane curing the machinery faults it met. That post was kept going through the whole test. Its working context was compacted once, at 12:09 UTC on 28 September 2026, after the edition was staged, and the same agent held the post across it: afterwards the post wrote the deposit metadata, completed its hand-back and passed its exit examination, set from its own record, in full agreement. A fresh hire of the same post then took up the final round. The census of Section 4 counts PM-post holdings only, so this post is evidenced by its own records: its run record in the supplement (`anc/post_run_record.json`), its exit examinations and its graded hand-backs. The rollout lane completed the census’s extension to posts hired as sub-agents on 28 September 2026, inside the window and before the next business test, on complex mathematical problems, begins; this paper’s figures come from the census as published, which counts PM-post holdings only. Splitting the work across more posts, or into more tasks, during advisory mode was a decision the authors could have made and did not; adding posts in the next test is a choice of speed and funding that we will evaluate separately.

Zero accepted after the compaction, with the caught count beside it. After the compaction, the project’s evidence check judged 19 turns and

its figure check 19 figures; 0 were caught and 0 were released. Across the whole test the two checks judged 32 turns and 32 figures, caught 1 and released 0.

Every fault the test filed was cured by the freeze. Between the agreement of the set-up, at 08:20 UTC on 28 September 2026, and the freeze of this pack at 14:55 UTC on 28 September 2026, the settled register gained 16 rulings and the defect record gained 45 faults, among them the one this paper’s own measurement caught (Section 4.6). All 26 filed on project 226 were landed and cured by the freeze. Of the 19 filed on the rollout lane, 15 were; the other 4 were faults in the rollout lane’s own machinery, found while it cured this test’s faults, and were still open on its record at the freeze. The list, by reference, is in the supplement with the freeze time. The set-up and the test both lie inside the evidence window.

7 The next problem

Can the architecture carry the same governed method into substantially harder reasoning?

The authors’ next ambition is to test whether IQRAX can preserve continuity, qualification, authority, and self-remediation while an AI model attacks a reasoning problem vastly harder than the business test above.

8 Verification and the supplement

The frozen supplement and every test tool is published. The operating state, the outcomes and the five further measures come from one census run over the window, through the classification tools’ own declared window: a row stamped at or after the freeze is not read. The census publishes no session identifier, plan identifier or message text, and quotes any message it classes as a correction only into separate files for the user’s review. Its 31 test cases, the 25 cases of the stricter user-input rule and the 6 of the tool that names the model behind the long turn each went red before the rule they prove was written, including the user’s refinement of the stricter rule; the two classification tools pass their own cases from the supplement (`anc/tool_p_roofs.json`).

Every reported aggregate is independently rederived from the frozen supplement. `anc/verify.py` checks the supplement against its

manifest and re-derives every aggregate in Section 4, the long turns’ dated inputs, and the post’s run in Section 6, from the frozen files, including every percentage, the per-day releases, the per-segment inputs, the cure triggers and the start-up ratios. A separate claim check reads every figure in this paper and matches it to a value in the supplement, a value the verifier derives, a date or a quotation; anything unmatched is an evidence gap and any figure stated two ways is a conflict. It reported 0 gaps and 0 conflicts in this edition, and its record, bound to the text it checked, is `anc/claim_check.json`. A release gate then rebuilds the paper from the source archive, runs the verifier with a planted wrong figure in every evidence file, and scans every file for confidential names before the edition is handed over.

9 Related work

Protection architecture keeps a governed resource outside the authority of the program seeking access: Saltzer and Schroeder [2] set out fail-safe defaults and complete mediation, and Schneider [3] characterises the policies execution monitoring can enforce. IQRAX applies an institutional version of that separation to role occupancy, action authority, accepted outcomes and repair admission, and does not claim the security guarantees of any earlier system. Reflexion [4] improves agents outside their weights through retained feedback; our object is who may admit a change to the institution, and on what evidence. Durable agent state is the closest neighbour: MemGPT [5] pages information between the context window and outer memory, and LangGraph [6] persists graph state as checkpoints. Those systems help an agent remember. IQRAX makes the enterprise remember: the record lives in the project on the enterprise’s device, it is captured at every act, and what it holds governs the work. TypeSafe AI’s Jev [7] narrows the probabilistic surface with typed, calibrated decisions, a welcome constraint on form and complementary to our question of who qualifies the decider, who accepts the decision, and where the record of both is kept.

10 Limitations

Scope. The results are observed on 20 operational holdings on three projects, two of them client projects and one the authors’ own publication project, held by 5 sessions over 6 days, 14 hours and

55 minutes, and on the rollout lane beside them. The census as published counts PM-post holdings only; a post hired as a sub-agent is evidenced by its own records, and the census’s extension to such posts, completed on 28 September 2026 before the next business test, does not enter this paper’s figures. The results are associations under the stated conditions, with no control group, and not isolated causal effects. The authors built, operated and measured the estate. The cohort predates the stamping of every verdict with the version of the control that wrote it, and the figure check’s verdicts carried no timestamp until 27 September: those we could not place in the window are counted apart, never dropped, and the 38 stamped verdicts since then are counted where they fell. The 3 holdings that cannot be established are reported as counted. The classification of each cure’s trigger and of each user input is a published text rule, not a reading of intent: it counts any quoted user ruling as the user’s trigger, and a message as a correction only when its words reverse or redirect the holder’s work or name a settled process the holder did not follow. The results therefore demonstrate operation under the measured estate and window; they do not establish equivalent performance across other models, organisations, workloads, or deployment environments.

11 Conclusion

The enterprise does not have to give probabilistic intelligence deterministic authority in order to use it. IQRAX separates the two. The enterprise-controlled project record carries the work across sessions and compactions; sealed qualification determines which agents may occupy its roles; deterministic authority decides which acts and outcomes may pass; and a refusal keeps the obligation open until the fault is remediated at source.

Across the frozen 6-day, 14-hour and 55-minute evidence window, the operational cohort answered 44 of 44 project-record questions, carried 18 of 18 successor holdings forward from banked judgment, left 0 of 93 surfaced faults open, and released no violation through either outcome check. Across the full estate, 246 of 323 cures were initiated without user intervention, while the business test carried a qualified post from advisory mode through a sealed plan and execution to a staged edition that passed the release gate.

Sessions ended and compacted; the project continued. Agents changed; the institutional record remained. Refusals occurred; the obligation remained open until resolved. That is the proposition tested here: probabilistic intelligence can be given room to reason while qualification, authority, acceptance, and institutional memory remain deterministic and outside the model.

Declarations

Availability. The supplement contains the frozen window census, the start-up pair, the two long-turn records and the model behind each, the independent reviewer’s signed statement, and the run record of the post that did this paper’s work; the measuring tools and the stricter user-input rule with their test cases; the figures’ drawing program; the claim check record; the claim-to-evidence map; a verification program; and a provenance manifest. Raw operational logs, client material, transcripts, and message text are not included.

Competing interest. The authors are independent researchers with a commercial interest in the architecture described. IQRAX is the name of the architecture, not of an employing or sponsoring organisation, and the study concerns the authors’ own development and operating estate.

Confidentiality. Client names, process names, and the names and qualifications of the roles used in the work are internal records whose publication has not been authorised. Projects appear by number only, posts by role class only, and plans by project and order.

AI assistance. Claude Code participated in system development, analysis, evidence preparation and the drafting of this paper.

References

- [1] Mohamed Dafea and Durre Shehwar Hanif. IQRAX: Deterministic Authority, Remediation at Source, and an Eight-Property Controllability Conformance Profile for Enterprise AI. Zenodo, preprint, 22 September 2026. <https://doi.org/10.5281/zenodo.22903272>
- [2] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278-1308, 1975. <https://www.mit.edu/~Saltzer/publications/protection/>
- [3] Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*,

3(1):30-50, 2000. <https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf>

- [4] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. <https://arxiv.org/abs/2303.11366>

- [5] Charles Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>

- [6] LangChain. LangGraph documentation: Persistence. Accessed 27 September 2026. <https://docs.langchain.com/oss/python/langgraph/persistence>

- [7] TypeSafe AI. Introducing System One Models & Jev. 15 September 2026. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>